SAVEETHA SCHOOL OF ENGINEERING

OPERATING SYSTEMS

ASSIGNMENT | CO1

EVALUATE AND JUSTIFY THE OPERATING-SYSTEM DESIGN OF A SMART HOSPITAL

Student Details	Information
Name	S. Maruthu Pandi
Register Number	192511162
Department	B.E. Computer Science and Engineering (Cyber Security)
Course	CSA0421 – Operating Systems
Assignment	Assignment 1 (Tasks 5)

TASK 5 – EXAMINE SYSTEM CALLS

5.1 Categories of system calls

Category	Purpose	Hospital example
Process control	Create, execute, wait for and terminate processes; manage signals and process attributes.	Start the database backup program, launch an imaging worker, wait for a child process.
File management	Create/open/read/write/seek/close files; inspect metadata and permissions.	Read a monitor configuration file or write a clinical audit log.
Device management	Request and release device access; read/write device interfaces; configure device parameters.	Read a sensor interface or communicate with an MRI acquisition controller.
Information maintenance	Get/set time, system data, process attributes and resource information.	Timestamp patient readings and inspect process/resource usage.
Communication	Create communication channels and send/receive data using pipes, sockets or shared memory.	Send monitor telemetry to a ward gateway or replicate database updates.
Protection	Set permissions, identities, ownership and access-control attributes.	Restrict database files to the database service account.

5.2 What happens when a program calls read()

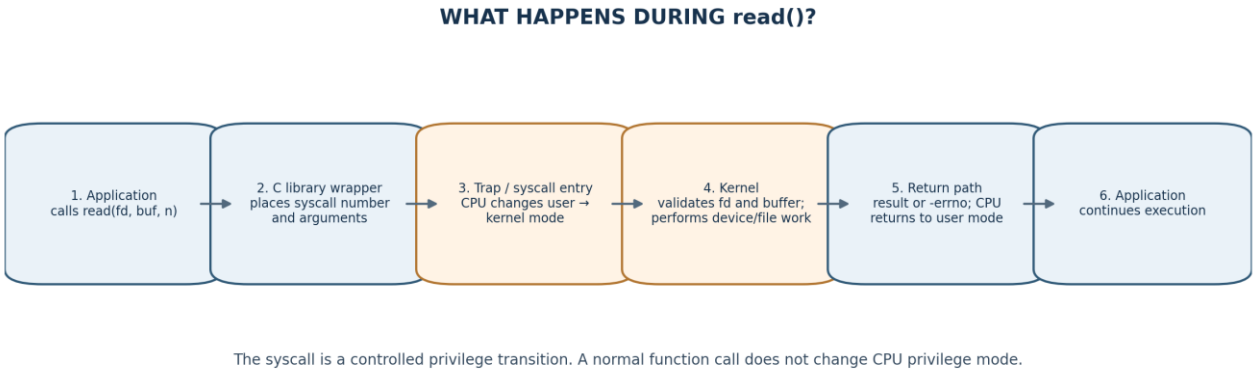


Figure 4. A read() request crosses from user mode to kernel mode and returns a byte count or an error.

1. The application calls the read() API, normally through a C library wrapper. The function arguments identify the file descriptor, destination buffer and maximum byte count.
2. The library wrapper places the system-call number and arguments in the architecture-defined registers or memory locations, then executes the syscall/trap instruction.
3. The CPU changes privilege from user mode to kernel mode and transfers control to the kernel system-call entry point. This is a controlled transition; the application cannot jump directly into arbitrary kernel code.
4. The kernel validates the file descriptor, access rights, buffer address and requested length. It invokes the filesystem, device driver or relevant cache path to obtain data.
5. The kernel places data in the caller's buffer when the operation completes, or records an error. The result is returned to user mode; a C library wrapper usually exposes an error through -1 and errno.
6. The application continues after read(). A successful call may return fewer bytes than requested, so robust programs must loop until the required data is received or end-of-file occurs.

5.3 How parameters are passed

Operating-system ABIs define how a system-call number and its arguments are passed. Common approaches include CPU registers, a pointer to a parameter block in memory, or stack-based arguments that are then copied or referenced by the wrapper. On modern 64-bit

Linux systems, the syscall number and a limited number of arguments are passed in designated registers according to the architecture ABI. The exact register names are architecture-specific, so a report should not assume that every CPU uses the same convention.

5.4 Observing calls with strace

strace intercepts and displays system calls made by a process. It is useful for checking whether a program uses openat(), read(), write(), close(), mmap() and related calls. It can also summarize call counts and time spent in system calls.

```
# Show selected system calls and their arguments
strace -e trace=openat,read,write,close ./read_demo records.dat

# Save the trace to a file
strace -o read_demo.strace ./read_demo records.dat

# Summarize system-call counts and time
strace -c ./read_demo records.dat
```

5.5 Small program for system-call evidence

```
#define _POSIX_C_SOURCE 200809L
#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>

int main(int argc, char **argv) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s file\n", argv[0]);
        return 1;
    }
    int fd = open(argv[1], O_RDONLY);
    if (fd < 0) { perror("open"); return 1; }

    char buf[4096];
    long total = 0;
    for (;;) {
        ssize_t n = read(fd, buf, sizeof buf);
        if (n == 0) break;
        if (n < 0) {
            if (errno == EINTR) continue;
            perror("read"); close(fd); return 1;
        }
        total += n;
    }
    if (close(fd) < 0) { perror("close"); return 1; }
    printf("Bytes read: %ld\n", total);
    return 0;
}
```

```
gcc -std=c11 -Wall -Wextra -O2 read_demo.c -o read_demo
./read_demo records.dat
strace -c -e trace=openat,read,close ./read_demo records.dat
```

Save the terminal output and strace summary in Task05/. Separate application read-loop calls from system calls made during program startup and library loading.

5.6 Alternative: C library fopen(), fread() and fwrite()

The C standard I/O functions fopen(), fread() and fwrite() operate on FILE streams and normally maintain a user-space buffer. When the application requests small pieces of data, the library can satisfy several requests from one filled buffer before making another read() system call. Similarly, fwrite() can collect many small writes and pass them to the kernel in larger blocks. Direct open()/read()/write() calls give the program lower-level control but can create many more system calls if the buffer is small.

For n bytes, a library buffer of about 4096 bytes needs roughly ceil(n/4096) reads, while a direct buffer of b bytes needs about ceil(n/b) successful reads plus EOF. Both process O(n) data; buffering reduces user/kernel transitions but uses extra user-space memory proportional to the stream buffer size